

Selecting the best ML architecture among LSTM, GRU, and Reservoir for small-size data

Tsuyoshi Yoneda
Hitotsubashi University

The foundation of modern AI: **Backpropagation**

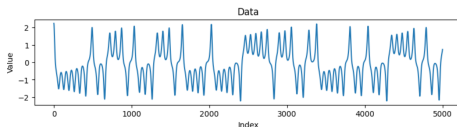
⇒ Vanishing gradient problem

- Hochreiter-Schmidhuber (1997)
Solved the vanishing gradient problem in RNNs by introducing **LSTM**
- Mikolov et al. (2010–2013), Sutskever-Vinyals-Le (2014)
LSTM began to be applied to natural language generation
- However, since 2017, **Transformers** (Vaswani et al., 2017) have become dominant, and LSTMs are no longer at the forefront

Personal perspectives and questions

- My research topic: Forecasting of stock price data and weather data
- **Small-size datasets (less than 10,000 samples)**
- For small-size datasets, is it really necessary to pursue the “context-level dependencies” that **LSTMs** and **Transformers** are good at?

Experimental setup for model comparison (1D Lorenz data)



Parameters fixed for fair accuracy comparison:

- Number of trainable parameters: 20,000
(nodes: Reservoir ~ 150 , LSTM ~ 70 , GRU ~ 80)
- Training data size: 5,000
- Prediction horizon: 100 steps ahead
- L^2 regularization parameter: 0.0001

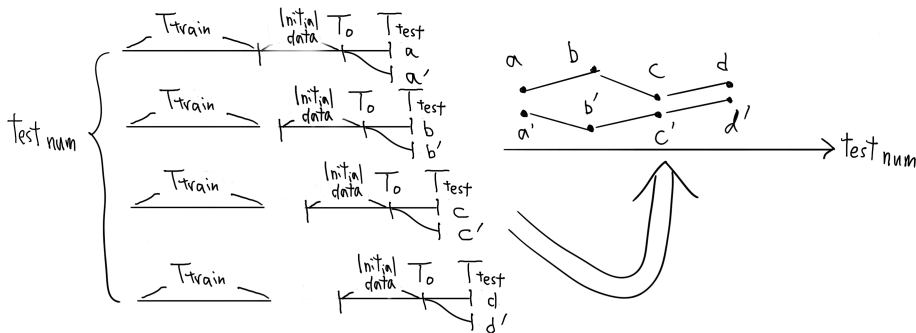
A low-spec machine on Google Colab (free version) is sufficient!

Candidate of ML architectures (3 models + 2 models)

- Standard LSTM implemented with TensorFlow (Adam optimizer for grad descent)
- LSTM with a novel gradient descent method without backpropagation
- GRU with a novel gradient descent method without backpropagation

Algorithm for model performance comparison

- Perform predictions **3000** times (test num), each shifted by one step, and compute the **MAE** (mean absolute error).
- Repeat the same process until the average **MAE** stabilizes (80 times).



The key point of novel grad descent

Delay Coordinates (see for e.g. Nakai-Saiki '21)

$$U_t = (\text{data}(t), \text{data}(t - \tau), \dots, \text{data}(t - (M - 1)\tau))$$

We train by using this U_t .

TensorFlow-LSTM: $\tau = 1$, M is chosen prior to training

Proposed Gradient Descent: τ and M are also learned during training

Sigmoid function h , $\odot$: element-wise product

LSTM

Input gate $i_t = h(W_{\text{datain}}U_t + W_{\text{gatein}}x_{t-1} + \text{bias}),$

Forget gate $f_t = h(W_{\text{dataforget}}U_t + W_{\text{forget}}x_{t-1} + \text{bias}),$

Output gate $o_t = h(W_{\text{dataout}}U_t + W_{\text{gateout}}x_{t-1} + \text{bias}),$

Memory cell $c_t = i_t \odot \tanh(W_{\text{datacell}}U_t + W_{\text{cell}}c_{t-1} + \text{bias}) + f_t \odot c_{t-1},$

Output $x_t = o_t \odot \tanh(c_t), \quad \hat{U}_{t+1} = W_{\text{out}}x_t$

Grad descent without Backpropagation (Bayes Opt)

Random matrices

$W_{datain}, W_{dataforget}, W_{dataout}, W_{datacell}, W_{gatein}, W_{forget}, W_{gateout}, W_{cell}$

By ridge regression (convexity), we first determine the following W_{out} :

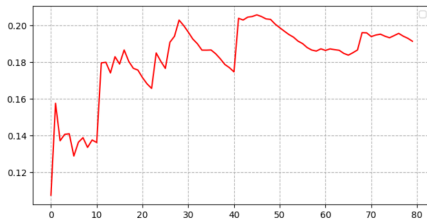
$$U_{t+1} \simeq W_{out}x_t \quad (\text{the idea of the conventional Reservoir}).$$

By the convexity (matrix computation), W_{out} is obtained. Note that we are not directly measuring the difference between U_{t+1} and the estimate $\hat{U}_{t+1}$ so far. In the hyper-parameter space, we apply Bayesian Optimization (Optuna) to minimize the following objective function (RMSE):

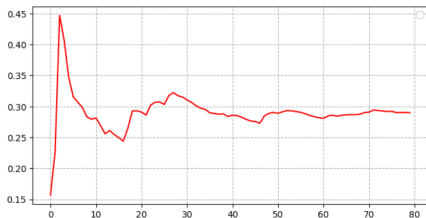
$$\left(\frac{1}{T} \sum_{t=0}^{T-1} |U_{t+1} - W_{out}x_t|^2 \right)^{1/2}.$$

Note: It is necessary to implement coding such that the random matrices vary continuously with continuous changes in the seed value!

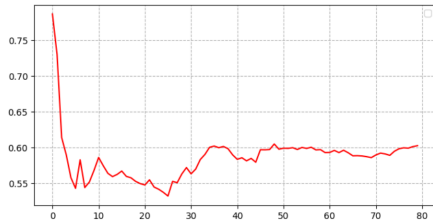
Results (GRU-Bayes, LSTM-Bayes, LSTM-TensorFlow)



GRU-Bayesian Optimization: MAE 0.19, 1h 30min



LSTM-Bayesian Opt: MAE 0.29,
1h 44min



LSTM-TensorFlow: MAE 0.60, 2h 59min

Refining ML Architectures

GRU performed better than LSTM

- ⇒ Couldn't lighter NN architectures produce better prediction results?
- ⇒ **Fast-lightweight LSTM**

The Bayesian optimization with delay coordinates performed better than the backpropagation method

- ⇒ Double-loop training scheme: **part of the training data is reserved as test data for model selection** and Bayesian opt is used for hyperparam tuning.
- ⇒ Since good prediction results can be easily imagined, reduce the data size to 1500+150 (training + model selection).

Final candidates of ML Architectures (2 models)

- **Fast-lightweight LSTM**
- **A reservoir for double-loop learning incorporating a model selection phase using Bayesian opt (online)**

Fast-lightweight LSTM (only cell layer) · probabilistic ESP

input gate: $i_t = \tanh(W_{datain}U_t)$

forget gate: $f_t = h(W_{dataforget}U_t + bias)$

memory cell: $c_t = i_t + f_t \odot c_{t-1}$

Ridge regression: $U_{t+1} \simeq W_{out}c_t$

$$\text{induction arg: } c_t = \sum_{s=0}^t \left(i_s \prod_{r=s+1}^t (\odot f_r) \right), \quad \prod_{r=t+1}^t f_r := 1.$$

Now assume that the following probability:

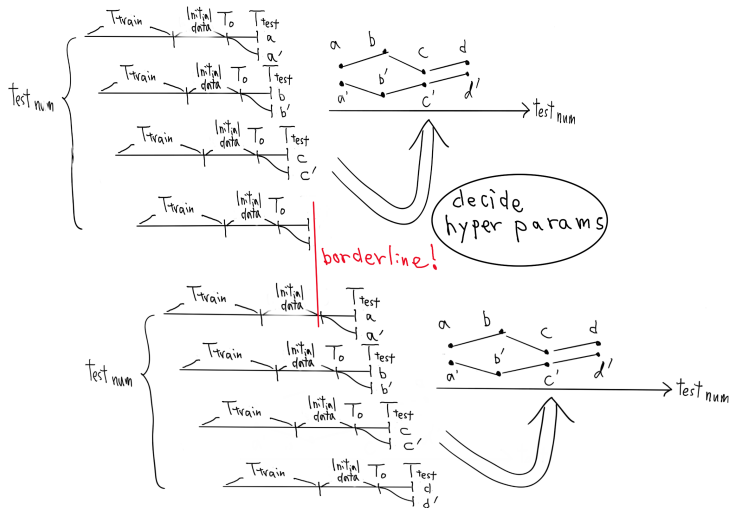
$$\gamma := P(f_r \cdot e_j \neq 0) \quad (0 < \gamma < 1)$$

is independent of j and r , where e_j is the j 'th unit vector. Also assume f_r and $f_{r'}$ ($r \neq r'$) are probabilistically independent. Then we have

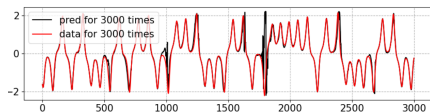
$$P\left(\prod_{r=s+1}^t f_r \cdot e_j \neq 0\right) = \prod_{r=s+1}^t P(f_r \cdot e_j \neq 0) = \gamma^{t-s} \rightarrow 0 \quad (s \ll t \rightarrow \infty).$$

Double-Loop Learning for Reservoir (Online)

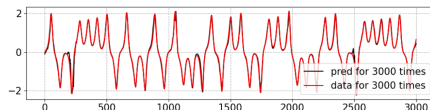
The first loop: Ridge regression, The second loop: Bayesian optimization using model selection data.



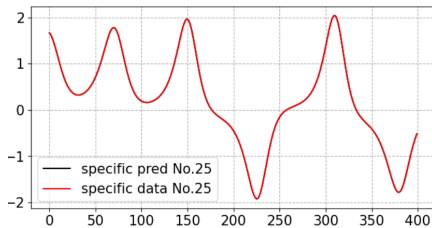
Both the fast-lightweight LSTM and the double-loop online Reservoir computing achieved good results



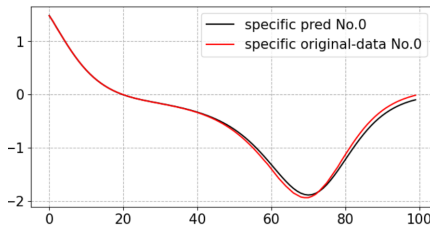
FastLightLSTM: MAE 0.11, prediction 400 steps ahead



Reservoir: MAE 0.067, 1500 training samples

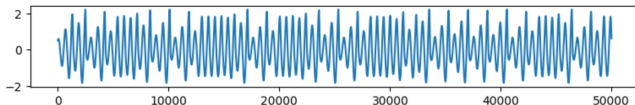


FastLightLSTM: MAE 0.11, prediction 400 steps ahead



Reservoir: MAE 0.067, 1500 training samples

Replication Test: Rössler Equation



- Number of trainable parameters: 20,000
(Reservoir ~ 150 nodes, LSTM ~ 70 , GRU ~ 80)
- Training data size: 5,000 (Reservoir uses 1,500+150)
- Prediction: 100 steps ahead (FastLightLSTM: 400 steps ahead)
- L^2 regularization parameter: 0.0001

Learning Results (MAE)

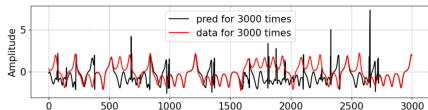
- | | |
|--|-------------------------|
| • FastLightweightLSTM: 0.005
(prediction 400 steps ahead) | • Bayes-LSTM: 0.055 |
| • DoubleRoopRes: 0.030
(1,500+150 training samples) | • Bayes-GRU: 0.061 |
| | • TensorFlow-LSTM: 0.52 |

Final judge: Lightweight LSTM vs Reservoir

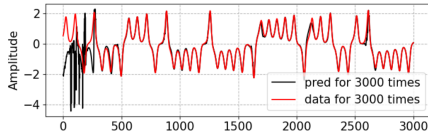
My research focuses on small-size real data.

⇒

For the final judge, we see that the training data is **more and more smaller-size** case.



LightLSTM: MAE 0.854, **650** train data



Res: MAE 0.188, **500+150** train data

- With only **650** data points, a significant difference emerged between the lightweight LSTM and reservoir computing. Future research should adopt **reservoir computing** as the optimal approach.